KỸ THUẬT LẬP TRÌNH AN TOÀN

Bài 02. Kiểm tra đầu vào

TÀI LIỆU THAM KHẢO

- Dr. David A. Wheeler, “Lecture: Secure Software Design & Programming”
- Jan Goyvaerts, “Regular expressions: The complete tutorial”

http://www.regular-expressions.info/print.html

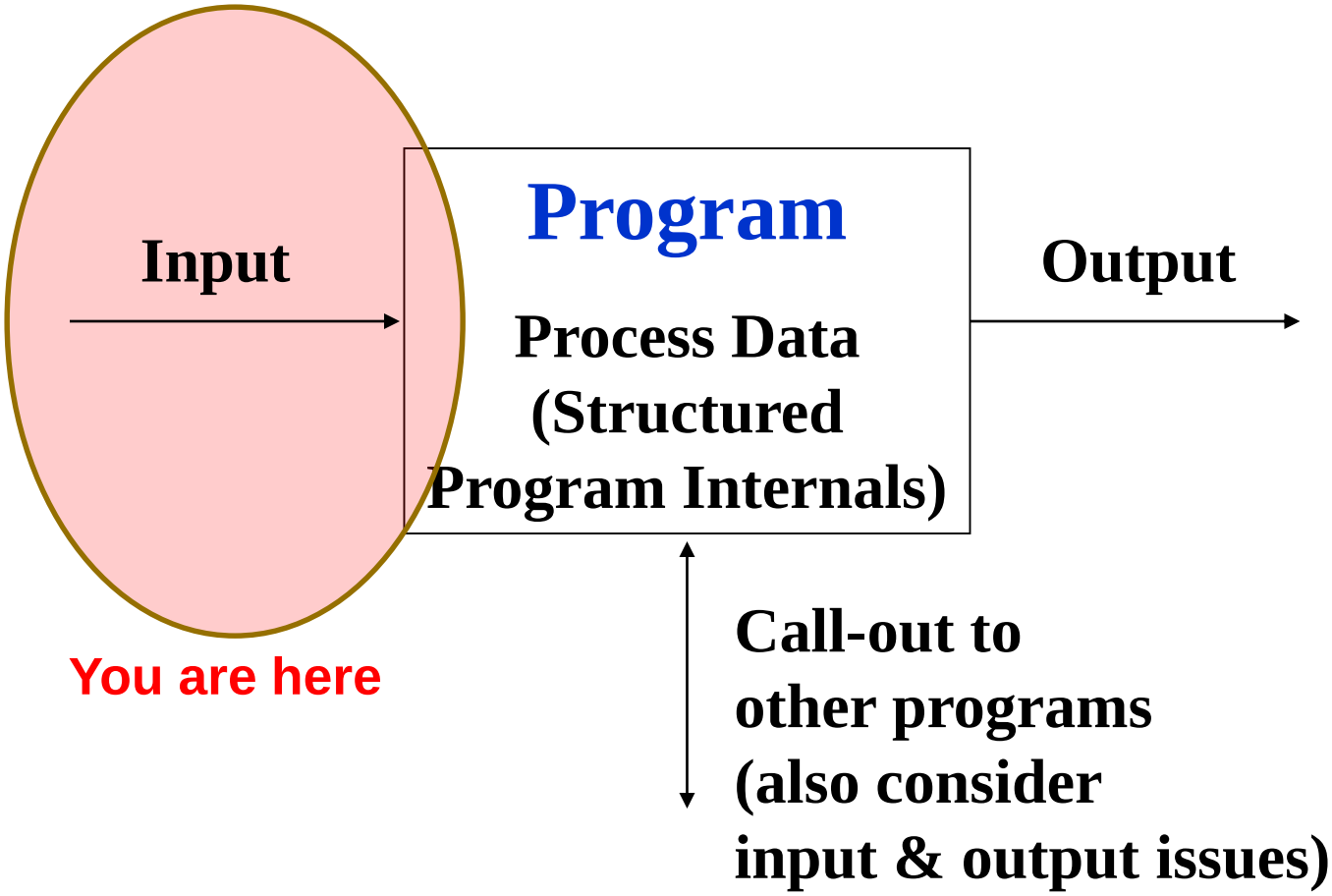
BÀI 02 – MỤC TIÊU

- Giúp SV nắm được tổng quan về kiểm tra đầu vào khi lập trình để đảm bảo an toàn
- Hướng dẫn SV cách sử dụng RE để kiểm tra đầu vào

NỘI DUNG

- Đầu vào
- Bề mặt tấn công
- Các nguồn đầu vào
- Khả năng không thể vượt qua
- Kiểm tra đầu vào
- Các kiểu đầu vào
- Biểu thức chính quy

Đầu vào


Input
Program
Process Data
(Structured
Program Internals)
Output
You are here
Call-out to
other programs
(also consider
input & output issues)

Bề mặt tấn công

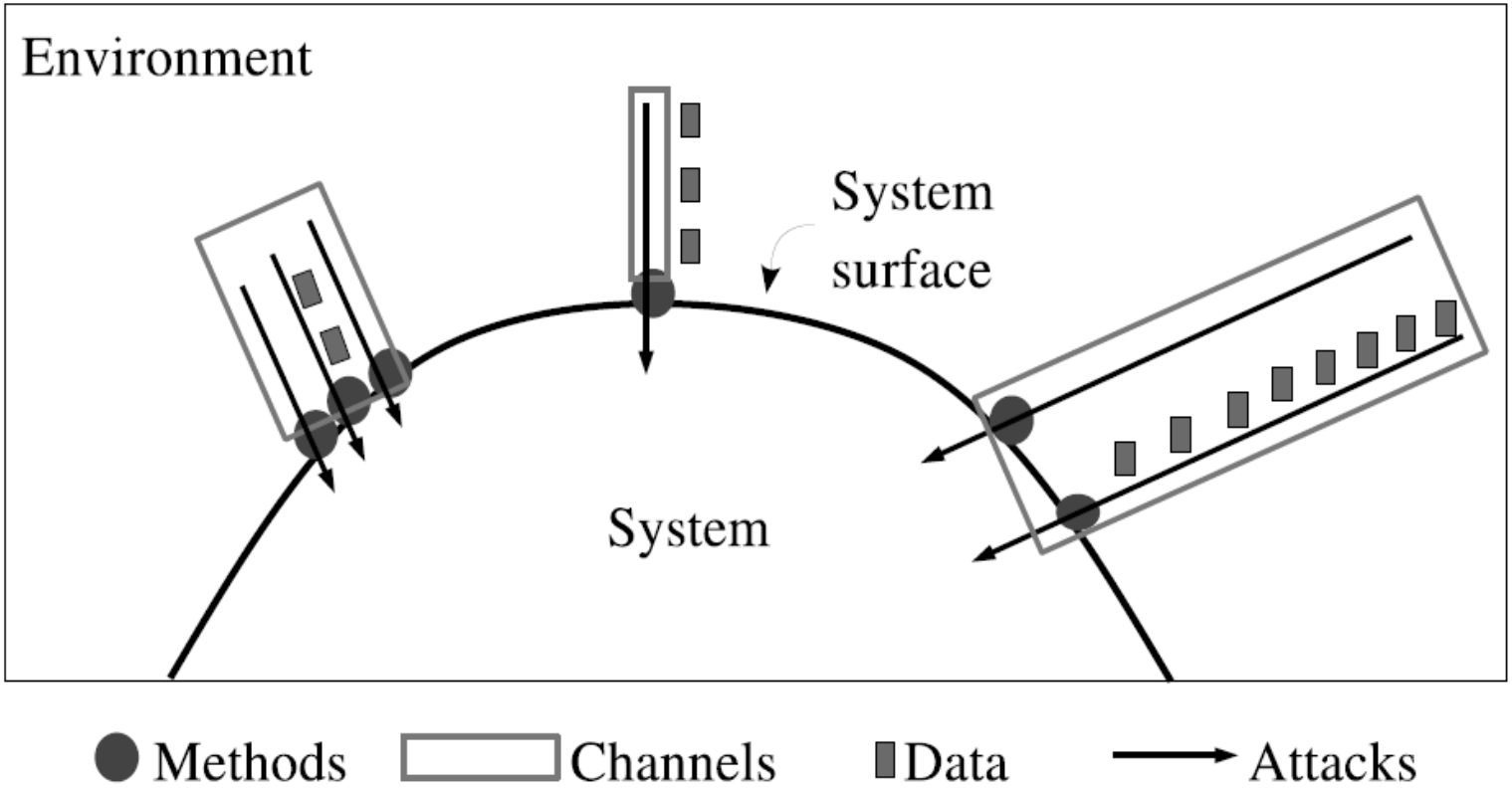
- Có thể tấn công qua:
  - Các kênh
    - Ports
  - Phương thức gọi (invoke methods)
    - API
  - Dữ liệu gửi đi
    - Các xâu đầu vào

Bề mặt tấn công

- Là tập con các nguồn tài nguyên của hệ thống (các kênh, phương thức và dữ liệu) mà có thể được dùng trong các tấn công lên hệ thống

Bề mặt tấn công

- Bề mặt tấn công càng lớn = càng dễ dàng khai thác và càng nguy hiểm



Bề mặt tấn công

- Người bảo vệ cần:
  - Làm bề mặt tấn công càng nhỏ càng tốt
    - Vô hiệu hóa các kênh và các phương thức
    - Ngăn chặn kẻ tấn công truy cập tới các kênh hoặc các phương thức
      - VD: Tường lửa
  - Nắm rõ các điểm đầu vào hệ thống
    - VD đối với mạng
      - Quét hệ thống để biết được

Bề mặt tấn công

- Với các bề mặt còn lại, cần làm càng sớm càng tốt:
  - Đảm bảo được xác thực và được ủy quyền (nếu phù hợp)
  - Đảm bảo kiểm tra tất cả các đầu vào không tin cậy
    - Đầu vào không tin cậy = bất cứ đầu vào nào từ một nguồn không hoàn toàn tin cậy
    - Các lỗi trong CWE-20: Improper Input Validation

Bề mặt tấn công

- Nhiều quan điểm: “kiểm tra tất cả đầu vào” chứ không chỉ các đầu vào không tin cậy
  - Ngay cả admin quan trọng cũng có thể gây ra lỗi
- => Kiểm tra đầu vào đối với tất cả các đầu không tin cậy cực kì quan trọng
  - Giúp kiểm soát nhiều tấn công

Bề mặt tấn công

- Chia nhỏ hệ thống
  - Là một kĩ thuật kiểm soát các tấn công
  - Chia thành những thành phần nhỏ hơn mà không thật sự đáng tin cậy với các thành phần khác
  - Mỗi thành phần nhỏ hơn có một bề mặt tấn công

Bề mặt tấn công

- Ngay cả với các ứng dụng web
  - Các tiến trình có thể được kẻ tấn công gọi
  - Có thể có một tiến trình có các quyền khác nhau

Các nguồn đầu vào

- Các nguồn đầu vào phụ thuộc vào loại ứng dụng, môi trường ứng dụng, …
  - Với các loại chương trình khác nhau:
    - Xác định một số kênh đầu vào và phương thức tiềm năng
      - Không giới hạn các kênh và phương thức chủ định
    - XĐ kẻ tấn công sẽ cố gắng thực hiện cái gì?
    - Xem xét nhiều loại hệ thống/môi trường/hệ điều hành khác nhau
      - VD: mobile app, web application, embedded device

Các nguồn đầu vào

Liệu có thể phát hiện ra bao nhiêu nguồn đầu vào không biết trước?

Các nguồn đầu vào

- Một số kênh tiềm năng
  - Dòng lệnh
  - Các biến môi trường
  - Kí hiệu mô tả file (file descriptors)
  - Tên file
  - Các đầu vào ứng dụng dựa trên Web
    - URL, POST, …

Các nguồn đầu vào

- Các loại đầu vào khác
  - Hệ thống cơ sở dữ liệu và các dịch vụ bên ngoài khác
  - Thuộc tính đăng kí/hệ thống
- => Chưa phải danh sách đầy đủ
  - Cần kiểm tra đầu vào đối với tất cả các kênh
    - Ít nhất đối với các kênh có đầu vào không tin cậy

Khả năng không thể vượt qua

- =Non-bypassability
- Là yêu cầu thực sự rất quan trọng
- Đảm bảo chắc chắn kẻ tấn công **không thể** vượt qua được kiểm tra
  - Tìm tất cả các kênh
    - Kiểm tra tất cả các đầu vào từ nguồn không đáng tin cậy từ các kênh
    - Kiểm tra càng sớm càng tốt

Khả năng không thể vượt qua

- Hệ thống client/server:
  - Thực hiện tất cả các kiểm tra liên quan tới an toàn tại server trong trường hợp thông thường

Khả năng không thể vượt qua

- VD HTML
  - Giả sử một ứng dụng gửi đoạn HTML này tới web browser như một phần của định dạng

```
<input name="lastname" type="text"
id="lastname" maxlength="100" />
```

  - Đoạn HTML này có cung cấp sự xác thực đầu vào liên quan đến an toàn không?
    - Ví dụ để đảm bảo lastname không dài hơn 100 kí tự?

Khả năng không thể vượt qua

- VD HTML (..)
  - Đoạn HTML trên không cung cấp bất cứ sự an toàn nào
    - Đoạn HTML được gửi tới web browser là client-side đã được định dạng và được xử lí
    - Điều này khiến nó không có giá trị đối với sự kiểm tra bypass và do đó thường là không liên quan tới an toàn

Khả năng không thể vượt qua

- VD HTML (..)
  - VD kẻ tấn công có thể viết một web browser client hoặc plug-in riêng
  - Đoạn HTML này có thể hữu ích để tăng tốc độ phản hồi mã không độc hại nhưng nó không chống lại tấn công

Kiểm tra đầu vào

- Whitelist >< blacklist
- Blacklist = mẫu xác định tất cả các đầu vào không được chấp nhận (tất cả các đầu vào khác được chấp nhận)
- Whitelist = mẫu xác định tất cả các đầu vào được chấp nhận (tất cả đầu vào khác bị từ chối)
- Dùng whitelist hay blacklist?

Kiểm tra đầu vào

- Một whitelist hoặc blacklist là một mẫu hoặc tập luật – không nhất thiết là một danh sách
- Không thực thi blacklist đối với kiểm tra đầu vào
  - Kẻ tấn công rất thông minh có thể luôn tìm được đầu vào "xấu" mới
  - Người dùng sẽ không cảnh báo bộ lọc không chặt chẽ

Kiểm tra đầu vào

- Thực thi kiểm tra đầu vào như một whitelist
  - Khiến kẻ tấn công chỉ có một số mẫu hạn chế có thể thực hiện
  - Nếu whitelist quá chặt chẽ, thì ít nhất người dùng sẽ thông báo

Kiểm tra đầu vào

- Dùng blacklist cũng được khi có thể liệt kê các mẫu có thể chứng minh
  - Tuy nhiên thực tế hiếm khi xảy ra

Kiểm tra đầu vào

- "Blacklist" hữu ích đối với kiểm tra
  - Xác định một số dữ liệu không được chấp nhận
    - Nhưng không dùng blacklist này như luật
  - Thay vào đó sử dụng blacklist để kiểm tra luật của whitelist
    - VD: sử dụng tập con của một blacklist như các trường hợp kiểm tra
    - Đảm bảo các luật của whitelist không chấp nhận blacklist

Kiểm tra đầu vào

- "Blacklist" hữu ích đối với kiểm tra (..)
  - Nói chung, các kiểm tra hồi quy cần kiểm tra xem "các hành động bị cấm" có thực sự là bị cấm không

=> Dùng whitelist, không dùng blacklist

Các kiểu đầu vào

- Số
- Chuỗi

Các kiểu đầu vào

- Số
  - Kiểm tra đối với các giá trị min và max
    - Đảm bảo chắc chắn tất cả các giá trị thuộc đúng phạm vi (tránh chia cho 0)
    - Với số nguyên không âm, sử dụng kiểu unsigned integer

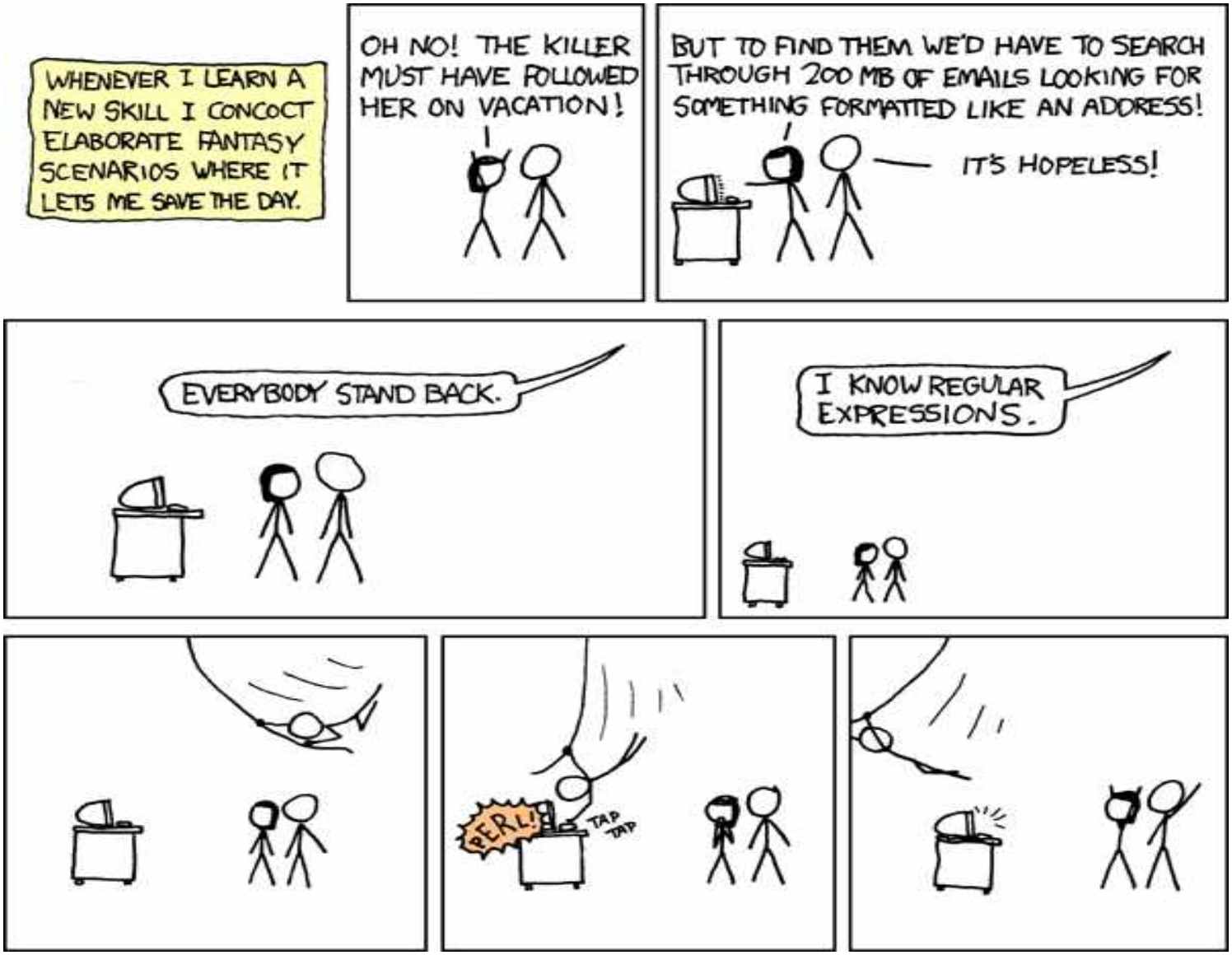
Các kiểu đầu vào

- Số
  - Có cho phép phân số?
    - Nếu không => sử dụng kiểu integer
  - Nếu có dấu phẩy động
    - Đề phòng các trường hợp lạ như NaN, Infinity, …

Các kiểu đầu vào

- Chuỗi
  - Nên đưa thành danh sách liệt kê nếu có thể
    - Đảm bảo chắc chắn chỉ một trong những giá trị trong danh sách mới là chính xác
  - Hạn chế độ dài tối đa
  - Kiểm tra có thỏa mãn luật whitelist không
    - "Đầu vào đúng sẽ luôn tuân theo whitelist"
    - Nếu kiểu chung (VD: địa chỉ email, URL, …) sử dụng lại luật
    - Dùng công cụ chung: biểu thức chính quy

Biểu thức chính quy: Giới thiệu



Source: Randall Munroe, "I know regular expressions", XKCD, http://xkcd.com/208/ Used by per permission granted at http://xkcd.com/license.html

Biểu thức chính quy: Giới thiệu

- =Regular expression (RE, regex, regexp)
- Định nghĩa
  - Là một mẫu mô tả một lượng văn bản được tìm kiếm trong một đoạn văn bản khác
  - VD: Tìm địa điểm học trong các buổi học
    - String=‘… Ngày 16-08-2021, lớp Kĩ thuật lập trình an toàn học tại P.203 TA2, … P.103 TA3, … P.509 TA1, …’
    - Pattern=‘/P.[0-9]{3}\sTA[123]/’
    - KQ: P.204 TA2, P.103 TA3, P.509 TA1

RE: Giới thiệu

- Nguyên tắc hoạt động
  - So khớp (match) dựa vào mẫu
    - Với mẫu được xây dựng từ các nguyên tắc của RE
  - Kết quả so khớp: có 2 giá trị
    - Khớp: Nếu tìm thấy một đoạn văn bản hoặc chuỗi byte hoặc chuỗi kí tự khớp (phù hợp) với mẫu
    - Không khớp: Ngược lại

RE: Giới thiệu

- VD: Tìm địa điểm học
  - String=‘Ngày 16-08-2021, lớp Kĩ thuật lập trình an toàn học tại P.203 TA2’
  - Với pattern=‘/P.[0-9]{3}\sTA[123]/’ ???
    - =>KQ: Khớp (P.203 TA2)
  - Nhưng với pattern=‘/T.[0-9]{3}\sTA[123]/’ ???
    - =>KQ: Không khớp

RE: Giới thiệu

- RE:
  - Có thể đơn giản
    - /a/, /2/, /[a-z]/,/ [0-9]/, /[a-z]+/, /[a-zA-Z0-9]*/, …
  - Nhưng cũng có thể rất phức tạp
    - /([a-zA-Z0-9.]+)@([a-z]+).([a-z]{2,5})/
    - /(^0[1-9]|[12][0-9]|3[01])\/(0[1-9]|1[012])\/(\d\d\d[1-9]|\d\d[1-9]\d|\d[1-9]\d\d|[1-9]\d\d\d)$/

RE: Giới thiệu

- Ứng dụng của RE
  - Được ứng dụng rộng rãi vì giúp tìm kiếm và thay thế nhanh hơn, dễ dàng hơn
  - Trước đây, RE được tạo ra để tìm kiếm văn bản
    - VD: Cho data và pattern

```
for position in 1..length(data):
    if
regex_match_at(pattern,data,position):
        return true
  return false
```

RE: Giới thiệu

- Ứng dụng của RE
  - VD: Cho data và pattern

```
for position in 1..length(data):
    if
regex_match_at(pattern,data,position):
        return true
  return false
```

    - => “abdicate” khớp với “ca[brt]”
      - Vì “cat” thuộc “abdicate”

RE: Giới thiệu

- Ứng dụng của RE
  - Hiện tại:
    - Tìm kiếm và thay thế trong các trình soạn thảo văn bản
      - VD: đối với trình soạn thảo Notepad++
    - **Ứng dụng trong các ngôn ngữ lập trình**
    - ...

RE: Lọc/kiểm tra/xác nhận đầu vào

- RE không phải là cách duy nhất để kiểm tra đầu vào
- RE thường là công cụ hữu ích đối với việc kiểm tra nhanh các đầu vào
  - Dễ sử dụng, sẵn có nhiều, linh hoạt, nhỏ gọn, thời gian thực thi thường nhanh, là chuẩn chung (được biết đến/được hiểu rộng rãi)
  - Nếu việc tạo kiểm tra đầu vào quá khó thì RE không thành công

RE: Lọc/kiểm tra/xác nhận đầu vào

- RE có thể được dùng để lọc đầu vào
  - Kiểm tra xem dữ liệu có khớp với mẫu không
- Với mỗi đầu vào văn bản, thường sẽ xác định một mẫu sử dụng RE
  - Mẫu mô tả đầu vào hợp lệ
  - Tạo mẫu càng hạn chế càng tốt

RE: Lọc/kiểm tra/xác nhận đầu vào

- Sau đó, khi nhận đầu vào, yêu cầu thư viện RE để kiểm tra mẫu có phù hợp với đầu vào không
  - Nếu không thì từ chối đầu vào

RE: Các quy tắc căn bản

- Khai báo chuỗi RE:

/ ↓ Bắt đầu

/ ↓ Kết thúc

  - VD: /biểu thức chính quy/
    - String=‘Đây là tiết học về biểu thức chính quy môn KTLTAT’
      - KQ: Khớp

RE: Các quy tắc căn bản

- Kí tự bắt đầu và kết thúc
  - Kí tự bắt đầu: ^
    - Ứng với sự bắt đầu của dữ liệu (hoặc dòng, theo tùy chọn)
  - Kí tự kết thúc: $
    - Ứng với sự kết thúc của dữ liệu (hoặc dòng, theo tùy chọn)
  - VD: /^abc$/
    - Ý nghĩa???

RE: Các quy tắc căn bản

- Kí tự bắt đầu và kết thúc
  - Với các mẫu “anchoring” (cố định)
    - Luôn đặt “^” vào đầu và “$” vào cuối của mẫu khi dùng RE để lọc đầu vào
    - VD: Thẻ cào, số điện thoại, tài khoản ngân hàng, …
  - Chú ý đối với ngôn ngữ Ruby
    - \A và \z để chỉ bắt đầu và kết thúc xâu
    - “^” và “$” ứng với bắt đầu và kết thúc dòng

RE: Các quy tắc căn bản

- Kí tự bắt đầu và kết thúc
  - Str1=‘MonKTLThay’
  - RE: /KTLT/
    - Khớp
  - str2=‘KTLT’
    - Khớp
  - Str1=‘MonKTLThay’
  - RE: /^KTLT$/
    - Không khớp
  - str2=‘KTLT’
    - Khớp

RE: Các quy tắc căn bản

- So khớp một kí tự đơn
  - Một kí tự A-Z, a-z, 0-9 (và nhiều kí tự khác) khớp với chính nó
    - Nó sẽ khớp với chữ hoa/chữ thường tương đương nếu cho phép lựa chọn “bỏ qua chữ hoa-thường” – không phải theo mặc định
  - “.” khớp với một kí tự bất kì

RE: Các quy tắc căn bản

- So khớp một kí tự đơn (..)
  - "\" là kí tự thoát (escape):
    - \char: có thể có ý nghĩa đặc biệt mà không phải char
      - \n khớp với dòng mới (thêm dòng)
      - \s: khớp với khoảng trắng
      - \d: khớp với [0-9]
    - \char: chính là nghĩa của char
      - \. khớp với dấu chấm (dot)
      - \[khớp với dấu ngoặc vuông trái
      - \\ khớp với một dấu gạch chéo ngược (backslash)

RE: Các quy tắc căn bản

- Phạm vi của chuỗi: []
  - Là dạng ngôn ngữ trong một ngôn ngữ
    - Ứng với **một** kí tự và cho phép thể hiện **một tập các kí tự** được chấp nhận trong []
    - Hai dạng:
      - [min-max], trong đó min, max lần lượt là kí tự bắt đầu và kết thúc của phạm vi
      - [list], trong đó list liệt kê các kí tự cho phép
    - VD: /[a-z]/; /[A-G]/ /[a-zA-Z0-9]/; /ca[brt]/

RE: Các quy tắc căn bản

- Phạm vi của chuỗi: []
  - Lưu ý đối với các kí tự đơn trong []: thường khớp với chính nó
    - Nhưng với dấu '–', VD [a-g-]
      - Dấu – đặt trước hoặc sau phạm vi hoặc \- không thuộc vào phạm vi
    - Dấu '^':
      - /[^a-z]/: phủ định
      - /[a-z^]/: là kí tự ^
    - Dấu '.': Không có nghĩa là một kí tự bất kì

RE: Các quy tắc căn bản

- Phạm vi của chuỗi: []
  - Phân biệt sự khác nhau khi dùng [] và không
    - VD: RE1=/ca[brt]; RE2=/cabrt/
      - ‘ca**b**’ hoặc ‘ca**r**’ hoặc ‘ca**t**’
        - Khớp với RE1 nhưng không khớp với RE2
      - ‘ca**brt**’
        - Không khớp RE1 nhưng khớp với RE2

RE: Các quy tắc căn bản

- Chiều dài có giới hạn của chuỗi RE: {min,max}, {N}
  - {min,max}, trong đó min, max lần lượt là chiều dài tối thiểu và tối đa của chuỗi RE
    - VD: RE=/[a-z]{3,5}/
      - ‘032ktl23’; ‘984ktlt98w’; ‘04ktlta23’; ‘54ktltatxyz32’ có khớp với RE???
        - => ‘032**ktl**23’; ‘984**ktlt**98w’; ‘04**ktlta**23’; ‘54**ktlta**txyz32’ khớp với RE
        - Đặc biệt nếu cho phép tìm tất cả các chuỗi khớp (thêm flag g) thì: ‘54**ktlta**txyz32’: có 2 chuỗi khớp với RE
      - Nhưng ‘032kt23’ không khớp với RE

RE: Các quy tắc căn bản

- Chiều dài có giới hạn của chuỗi RE
  - {N}: độ dài chính xác là N
    - VD: RE= /[0-9]{3}/
      - ‘ab0956ds’ có khớp với RE???
        - => ‘ab**095**6ds’ khớp với RE
      - ‘ab094387ds9874;234;34’ có khớp với RE???
        - Có mấy chuỗi khớp???
        - ‘ab**094**387ds**987**4;**234**;34’: có 4 chuỗi khớp với RE
      - ‘ab09ds’ có khớp với RE???
        - Không

RE: Các quy tắc căn bản

- Chiều dài không giới hạn của chuỗi RE
  - {N,}
  - *
  - +
  - ?

RE: Các quy tắc căn bản

- Chiều dài không giới hạn của chuỗi RE
  - {N,}: độ dài $\geq N$
    - VD: RE=/[0-9]{3,}/
      - ‘ab094387ds9874;234;34’: có khớp với RE???
        - => ‘ab**094387**ds**987**4;**234**;34’: có 3 chuỗi khớp với RE
      - ‘ab09ds’ có khớp với RE???
        - Không!

RE: Các quy tắc căn bản

- Chiều dài không giới hạn của chuỗi RE
  - *: 0 hoặc nhiều kí tự, tương đương với {0,}
  - Chuỗi có độ dài 0 là chuỗi rỗng
    - Cần lưu ý khi trả lại kết quả là chuỗi rỗng, nhiều trường hợp sẽ dẫn tới vô số kết quả
      - Không nên dùng chuỗi độ dài 0
  - VD: RE=/[a-z]*/ hoặc /[a-z]{0,}/
    - Str=‘4abcds9eddg3;a4’
      - => có 9 chuỗi khớp với RE

RE: Các quy tắc căn bản

- Chiều dài không giới hạn của chuỗi RE
  - +: 1 hoặc nhiều kí tự, tương đương với “{1,}
  - VD: RE=/[a-z]+/ hoặc /[a-z]{1,}/
    - Str= ‘4abcds9eddg3;a’ có khớp với RE???
      - => ‘4**abcds**9**eddg**3;**a**’ có 3 chuỗi khớp với RE

RE: Các quy tắc căn bản

- Chiều dài không giới hạn của chuỗi RE
  - ?: 0 hoặc 1 kí tự, tương đương với “{0,1}
  - VD: RE=/[a-z]?/ hoặc /[a-z]{0,1}/
    - => ‘4a9eg3;a’ có 9 chuỗi khớp với RE

{N}; {N,}; {min,max}; *; +; ? được gọi là **dấu hiệu lặp** (duplication marker)

RE: Các quy tắc căn bản

- RE đại diện cho **một** kí tự bất kì:
  - Sử dụng: .
  - VD1: RE=/.{2,5}/
    - ‘a3v@2!~$#d/,.<+)’ có khớp với RE???
      - =>‘a3v@2**!~$#d/,.<+**)’ có 3 chuỗi khớp
  - VD2: RE= /^.{2,5}$/
    - ‘a’’; ‘a3v@2’; ‘a3v@2d’ có khớp với RE???
      - ‘a3v@2’ khớp nhưng ‘a’ hay ‘a3v@2d’ không khớp
        - Vì kí tự ^ và $ giới hạn độ dài chính xác từ 2-5

RE: Các quy tắc căn bản

- Kí tự đơn, ‘.’, ‘\char’ và biểu thức ngoặc vuông gọi là phần tử cơ bản (atom)
- Một phần tử cơ bản có thể kèm theo dấu hiệu lặp và được gọi là một bộ phận (piece)
  - VD:
    - /a{4,6}/
    - /.{3}/
    - /\^{2,}/
    - /[abc]+/

RE: Các quy tắc căn bản

- VD: RE=/ [abc]+/
  - Ý nghĩa???
  - => “1 hoặc nhiều các kí tự a, b hoặc c”
    - => ‘a’, ‘aaeaa’, ‘dcabe2’, ‘abefba’, ‘pccbb23aac02’ **khớp** với RE
    - Nhưng ‘dog’ hoặc ‘235’ hoặc ‘de043’ **không khớp** với RE
    - ‘A’ **không khớp** với RE trừ khi yêu cầu so khớp **không phân biệt chữ hoa - chữ thường**

RE: Các quy tắc căn bản

- VD: RE=/ ^[abc]+$/
  - Ý nghĩa???
  - => “1 hoặc nhiều các kí tự a, b hoặc c nhưng không chứa các kí tự khác ngoài a, b và c”
    - => ‘a’, ‘aaaa’, ‘cab’, ‘abba’, ‘ccbbaaac’ **khớp** với RE
    - Nhưng ‘dog’ hoặc ‘ad’ hoặc ‘a$’ **không khớp** với RE
    - ‘A’ **không khớp** với RE trừ khi yêu cầu so khớp **không phân biệt chữ hoa - chữ thường**

RE: Các quy tắc căn bản

- Gom nhóm RE
  - Có thể gom nhóm các biểu thức RE với ()
    - Biến toàn bộ biểu thức thành một phần tử cơ bản
    - Một khi thực hiện như vậy, có thể thêm vào sau nó một ràng buộc
      - VD tên file với quy ước đặt tên: “1 tới 8 kí tự số-chữ, tùy chọn thêm vào sau một đoạn có độ dài 2 đến 3 bắt đầu bằng dấu chấm rồi đến kí tự số-chữ”
        - Như RE: ^[a-zA-Z0-9]{1,8}(\.[a-zA-Z0-9]{2,3})?$

RE: Các quy tắc căn bản

- Gom nhóm RE
  - Giúp dễ nhìn RE hơn
  - Nhưng kết quả trả về của các chuỗi so khớp có thay đổi

RE: Các quy tắc căn bản

- RE A hoặc B
  - Có thể liệt kê các biểu thức lựa chọn, được phân tách bởi “|”; bất kì lựa chọn nào sau đó có thể phù hợp
    - Mỗi lựa chọn được gọi là một “nhánh”
  - VD: RE=/[a-z]|[0-9]/, /[a-z]{3}|[0-9]/, /^[a-z]{3}|[0-9]$/, …

RE: Các quy tắc căn bản

- RE A hoặc B
- "|" có mức độ ưu tiên thấp hơn "^" hoặc "$"
  - Vì vậy thường phải đặt ngoặc đơn là (…|…)
  - Trong các phép lọc cần phải sử dụng "|" trong (…)
  - VD: RE1= /^cat|bird$/; RE2= /^(cat|bird)$/
    - Bất cứ cái gì bắt đầu với cat hoặc bất cứ cái gì kết thúc là bird khớp với RE1
    - Chỉ 'cat' hoặc 'bird' khớp với RE2

RE: Các quy tắc căn bản

- RE phủ định
  - Sử dụng: [^…]
  - VD: RE=/[^a-g]/; /[^0-9]/; /[^a-z0-9]/
  - Chú ý phân biệt:
    - ^ khi là kí tự bắt đầu
      - VD: RE=/^[a-z]/
    - ^ khi là phủ định
      - VD: /[^a-z]/

RE: Các quy tắc căn bản

- Capturing value trong RE
  - VD: RE=/([A-Z]+)**([0-9]+)**/
    - Phân tích thành 3 phần RE:
      - Phần 1: Toàn bộ RE ([A-Z]+)([0-9]+)
      - Phần 2: RE con đầu tiên ([A-Z]+)
      - Phần 3: RE con cuối ([0-9]+)
        - => 3 phần này gọi là **Capturing Value** trong RE

RE: Các quy tắc căn bản

- Greedy trong RE
  - VD: Tìm chuỗi bắt đầu bằng chữ ‘K’ và kết thúc bằng ‘AT’ trong chuỗi ‘KTLT-AT CHIA THANH 2 LOP KT-LTAT 01 VA KT-LT-AT 02’
  - => RE=/K(.+)AT/
    - KQ

```
Array
(
 [0] => KTLT-AT CHIA THANH 2 LOP KT-LTAT 01 VA KT-LT-AT
 [1] => TLT-AT CHIA THANH 2 LOP KT-LTAT 01 VA KT-LT-
)
```

RE: Các quy tắc căn bản

- Greedy trong RE
  - VD
    - Thấy rằng: ‘**KTLT-AT** CHIA THANH 2 LOP KT-LTAT 01 VA KT-LT-AT 02’
      - Làm sao đưa ra kết quả là chuỗi đầu tiên?
      - => **Chống greedy**
        - Đặt dấu ? Vào sau các RE
        - VD: RE=/K(.+?)AT/

RE: Các quy tắc căn bản

- Các kí tự đặc biệt
  - Là các kí tự dấu chấm ., ngoặc đơn (), ngoặc vuông [] hoặc những kí tự liên quan tới quy tắc của RE

RE: Các quy tắc căn bản

- Các kí tự đặc biệt
  - VD: Kiểm tra một chuỗi có tồn tại dấu chấm không?
    - Chuỗi: ‘Day la tiet hoc KTLTAT.’
      - =>RE=?
      - Làm sao để không hiểu . không là một kí tự bất kì?
  - **Giải pháp**???

➡ **Thêm dấu \ vào trước kí tự đặc biệt**

RE: Các quy tắc căn bản

- Các kí tự đặc biệt
  - Danh sách một số kí tự đặc biệt

Kí tự	Ý nghĩa	Kí tự	Ý nghĩa
.	Kí tự bất kì ngoại trừ kí tự ngắt dòng và kí tự có dấu (unicode)	^	Phủ định (khai báo [^…])
|	Quan hệ hoặc	\d	[0-9]
()	Gom nhóm	\D	[^0-9]
[]	Phạm vi	\w	[a-zA-Z0-9_]: các kí tự là từ
*	Không có hoặc nhiều kí tự	\W	[^a-zA-Z0-9_]: các kí tự không phải là từ
+	1 hoặc nhiều kí tự	\s	Một kí tự khoảng trắng

RE: Các quy tắc căn bản

- Các kí tự đặc biệt
  - Danh sách một số kí tự đặc biệt

Kí tự	Ý nghĩa	Kí tự	Ý nghĩa
?	Không có hoặc 1 kí tự/nhóm kí tự () trước dấu ?	\S	Một kí tự bất kì không phải khoảng trắng
^…	Bắt đầu của dòng	…$	Kết thúc của dòng
…\b	Kết thúc trong các từ bằng xâu … => có thể có nhiều kết quả (khác với $ kết thúc của cả xâu=> chỉ 1 kết quả nếu có)	\b…	Bắt đầu các từ bằng xâu … => có thể có nhiều kết quả (khác với ^ bắt đầu của cả xâu=> chỉ 1 kết quả nếu có)
…\B	Không kết thúc bằng xâu trước \B	\b…\b	Bắt đầu và kết thúc của 1 từ là xâu … => có thể có nhiều kết quả (khác với ^ …$ chỉ có 1 kết quả nếu có)

RE: Các quy tắc căn bản

- VD AD
  - 1. Tìm xâu ‘Lop KTLT’ trong văn bản
  - 2. Tìm các chữ in thường trong văn bản
  - 3. Kiểm tra trường nhập tên người dùng gồm: kí tự chữ thường, số, gạch dưới, gạch nối, dấu chấm và có độ dài từ 5-15 kí tự
  - 4. Kiểm tra trường nhập số điện thoại

RE: Negative & Positive lookbehind

- Lookbehind: Xác định sự lựa chọn ở phía trước của một chuỗi RE
  - Gồm 2 loại:
    - Positive lookbehind
    - Negative lookbehind

RE: Negative & Positive lookbehind

- VD1: Tìm tất cả phần đuôi email của ‘ktltat’ trong chuỗi sau:
  - ‘**ktltat**@actvn.edu.vn; ktltat01@actvn.edu.vn; **ktltat**@facebook.com; ktltat02@gmail.com; **ktltat**@gmail.com; ktltat01@gmail.com; ktltat02@facebook.com’
  - => RE=???
    - RE=/@[^;]+/???

RE: Negative & Positive lookbehind

RE = /@[^;]+/

```
string=ktltat@actvn.edu.vn; ktltat01@actvn.edu.vn;
       ktltat@facebook.com; ktltat02@gmail.com;
       ktltat@gmail.com; ktltat01@gmail.com;
       ktltat02@facebook.com
Các đuôi email của ktltat là:
Array
(
  [0] => Array
   (
    [0] => @actvn.edu.vn
    [1] => @actvn.edu.vn
    [2] => @facebook.com
    [3] => @gmail.com
    [4] => @gmail.com
    [5] => @gmail.com
    [6] => @facebook.com
   )
)
```

???

RE: Negative & Positive lookbehind

- => Làm sao để chỉ lấy đuôi email của ktltat???
  - RE=/ktltat@[^;]+/???

RE: Negative & Positive lookbehind

RE = /ktltat@[^;]+/

```
string=ktltat@actvn.edu.vn; ktltat01@actvn.edu.vn;
       ktltat@facebook.com; ktltat02@gmail.com;
       ktltat@gmail.com; ktltat01@gmail.com;
       ktltat02@facebook.com
Các đuôi email của ktltat là:
Array
(
  [0] => Array
   (
    [0] => ktltat@actvn.edu.vn
    [1] => ktltat@facebook.com
    [2] => ktltat@gmail.com
   )
)
```

THỪA!!!

RE: Negative & Positive lookbehind

- Giải pháp nào mới đúng???
  - => Dùng **Positive lookbehind**

Cú pháp: **(?<=)**

- => Giải pháp cho VD1
  - RE=/(?<=ktltat)@[^;]+/

RE: Negative & Positive lookbehind

RE = /(?<=ktltat)@[^;]+/

```
string=ktltat@actvn.edu.vn; ktltat01@actvn.edu.vn;
       ktltat@facebook.com; ktltat02@gmail.com;
       ktltat@gmail.com; ktltat01@gmail.com;
       ktltat02@facebook.com
Các đuôi email của ktltat là:
Array
(
  [0] => Array
    (
      [0] => @actvn.edu.vn
      [1] => @facebook.com
      [2] => @gmail.com
    )
)
```

=> CHÍNH XÁC

RE: Negative & Positive lookbehind

- RE=/(?<=ktltat)@[^;]+/

  - Là đoạn Positive lookbehind
  - Tức là **bắt đầu** bằng ktltat nhưng sẽ **không có trong kết quả** trả về

RE: Negative & Positive lookbehind

- VD2: Tìm tất cả phần đuôi của email **không phải** là ktltat của chuỗi trong VD1
  - => Dùng **Negative lookbehind**

Cú pháp: **(?<!)**

  - RE=/(?<!ktltat)@[^;]+/

RE: Negative & Positive lookbehind

string = ktltat@actvn.edu.vn; ktltat01@actvn.edu.vn; ktltat@facebook.com; ktltat02@gmail.com; ktltat@gmail.com; ktltat01@gmail.com; ktltat02@facebook.com

```
Các đuôi email không của ktltat là:
Array
(
    [0] => Array
        (
            [0] => @actvn.edu.vn
            [1] => @gmail.com
            [2] => @gmail.com
            [3] => @facebook.com
        )
)
```

RE: Negative & Positive lookahead

- VD1: Tìm trong chuỗi sau tất cả **phần đầu** email mà có phần đuôi là **@facebook.com**:
  - ‘ktltat@actvn.edu.vn; ktltat01@actvn.edu.vn; **ktltat**@facebook.com; ktltat02@gmail.com; ktltat@gmail.com; ktltat01@gmail.com; **ktltat02**@facebook.com’
  - =>Dùng **Positive lookahead**
  - => RE=???
    - RE=/[^;]+(?=@facebook\.com)/

RE: Negative & Positive lookahead

RE = /[^;]+(?=@facebook.com)/
string = ktltat@actvn.edu.vn; ktltat01@actvn.edu.vn; ktltat@facebook.com; ktltat02@gmail.com; ktltat@gmail.com; ktltat01@gmail.com; ktltat02@facebook.com

```
Phần đầu các chuỗi có đuôi email là facebook.com:
Array
(
    [0] => Array
        (
         [0] =>  ktltat
         [1] =>  ktltat02
        )
)
```

RE: Negative & Positive lookahead

- VD2: Tìm trong chuỗi sau tất cả **phần đầu** email mà có phần đuôi không phải là **@facebook.com**:
  - ‘ktltat@actvn.edu.vn; ktltat01@actvn.edu.vn; **ktltat**@facebook.com; ktltat02@gmail.com; ktltat@gmail.com; ktltat01@gmail.com; **ktltat02**@facebook.com’
  - =>Dùng **Negative lookahead** => RE=???

RE: Một số mẫu

- Khớp bất kì cái gì, ngoại trừ kí tự dòng mới nhúng
  - ^.*$
    - => Không sử dụng mẫu này để lọc
- Chuỗi gồm bất kì kí tự nào với độ dài từ 0 tới 12:
  - ^.{0,12}$
    - => Không nên dùng

RE: Một số mẫu

- Ngày theo dạng yyyy-mm-dd ???
  - ^[1-9][0-9]{3}-[01]?[0-9]-[0-3]?[0-9]$
    - Với ngày 2011-09-12??? ➡ **ĐÚNG**
    - Nhưng với ngày:
      - “1000-00-00, 9999-19-39”???

⬇

???

RE: Một số mẫu

- Lọc ngày tốt hơn đối với yyyy-mm-dd
  - ^[1-9][0-9]{3}-(0[1-9]|1[0-2])-(0[1-9]|[12][0-9]|3[0-1])$
  - Chấp nhận 2011-09-12
  - Không chấp nhận 1000-00-00, 9999-99-99
  - Nhưng chấp nhận 2011-02-31 và không chấp nhận 0001-09-30

RE: Một số mẫu

- Lọc ngày tốt hơn nữa đối với yyyy-mm-dd
  - ^(000[1-9]|00[1-9][0-9]|0[1-9][0-9]{2}|[1-9][0-9]{3})-(0[1-9]|1[0-2])-(0[1-9]|[12][0-9]|3[0-1])$
    - Chấp nhận năm từ 0001-9999; tháng từ 01-12; ngày từ 01-31
    - Nhưng tháng 2 và các tháng có 30 ngày vẫn chấp nhận ngày 31
      - Khó có thể xử lí điều này với RE
      - Sử dụng RE để loại bỏ hầu hết các trường hợp rồi sử dụng code đối với các kiểm tra ngữ nghĩa cụ thể
      - Nên sử dụng lịch để chọn

RE: Một số mẫu

- Số nguyên không âm có độ dài tối đa là 20 chữ số
  - ^(0|[1-9][0-9]{0,19})$
    - Tại sao dùng “|” ???
      - => Ngăn chặn số 0 đứng đầu

RE: Một số mẫu không tốt

- Định dạng ngày lộn xộn
  - ^[1-9][0-9]{3}-(0[1-9]|1[0,1,2])-([0,1,2][1-9]|3[0-1])$
  - Khớp với 2011-11-12
  - Cũng khớp với 2011-1,-,1
  - “,” trong một biểu thức dấu ngoặc khớp với “,” chứ không phải là sự ngăn cách

RE

- RE rất hữu ích
- Một lập trình viên nên nắm rõ RE
- Nhưng RE không phải lúc nào cũng hiệu quả

Bài tập

- 1. Kiểm tra license dạng số xxxx.xxxxx.xxxx
- 2. Kiểm tra license dạng số, chữ hoa-thường xxxx.xxxxx.xxxx
- 3. Kiểm tra mật khẩu có dạng: Có thể là chữ hoa-thường hoặc số với độ dài tối thiểu là 8, tối đa là 15

Bài tập

- 4. Hãy kiểm tra mật khẩu với yêu cầu:
  - Bao gồm kí tự đặc biệt, số, chữ hoa-thường và dấu gạch dưới (_)
  - Kí tự đầu tiên là kí tự đặc biệt
  - Chiều dài từ 4-12 kí tự